

Lab program-18:

18. Construct a C program to simulate producer consumer problem using semaphores.

Code:

```
#include <stdio.h>
#include <windows.h>

#define BUFFER_SIZE 5
#define MAX_ITEMS 10

int buffer[BUFFER_SIZE] = {0};

HANDLE empty, full;

int produced_items = 0;
int consumed_items = 0;

DWORD WINAPI producer(LPVOID arg)
{
    while (produced_items < MAX_ITEMS)
    {
        WaitForSingleObject(empty, INFINITE);

        // Critical Section: Add item to buffer
        for (int i = 0; i < BUFFER_SIZE; i++)
        {
            if (buffer[i] == 0)
            {
                buffer[i] = produced_items + 1;
                printf("Produced: %d\n", buffer[i]);
                produced_items++;
                break;
            }
        }

        ReleaseSemaphore(full, 1, NULL);

        Sleep(1000);
    }

    return 0;
}

DWORD WINAPI consumer(LPVOID arg)
```

```

{
    while (consumed_items < MAX_ITEMS)
    {
        WaitForSingleObject(full, INFINITE);

        // Critical Section: Remove item from buffer
        for (int i = 0; i < BUFFER_SIZE; i++)
        {
            if (buffer[i] != 0)
            {
                printf("Consumed: %d\n", buffer[i]);
                buffer[i] = 0;
                consumed_items++;
                break;
            }
        }

        ReleaseSemaphore(empty, 1, NULL);

        Sleep(2000);
    }

    return 0;
}

int main()
{
    HANDLE producer_thread;
    HANDLE consumer_thread;

    // Create semaphores
    empty = CreateSemaphore(
        NULL,
        BUFFER_SIZE,
        BUFFER_SIZE,
        NULL
    );

    full = CreateSemaphore(
        NULL,
        0,
        BUFFER_SIZE,
        NULL
    );

    if (empty == NULL || full == NULL)
    {
        printf("Semaphore creation failed.\n");
    }
}

```

```

        return 1;
    }

    // Create threads
    producer_thread = CreateThread(
        NULL,
        0,
        producer,
        NULL,
        0,
        NULL
    );

    consumer_thread = CreateThread(
        NULL,
        0,
        consumer,
        NULL,
        0,
        NULL
    );

    // Wait for threads to finish
    WaitForSingleObject(producer_thread, INFINITE);
    WaitForSingleObject(consumer_thread, INFINITE);

    // Cleanup
    CloseHandle(producer_thread);
    CloseHandle(consumer_thread);
    CloseHandle(empty);
    CloseHandle(full);

    return 0;
}

```

OUTPUT:

```

PS C:\Users\SMILE\Documents\OS\Lab programs> ./lab_18.exe
Produced: 1
Consumed: 1
Produced: 2
Consumed: 2
Produced: 3
Produced: 4
Consumed: 3
Produced: 5
Produced: 6
Consumed: 5
Produced: 7

```

